Explicación técnica completa

Proyecto Apertura — DBSCAN paralelo con OpenMP — código, función por función y parámetro por parámetro

José y Natalia

Octubre 2026

Contents

Propósito de este documento **2**

El algoritmo, explicado a fondo **2**
Qué es un punto "core" y qué es "ruido" . . . 2
Por qué es O(n²) . . . 2

`src/dbscan_common.h` **— código compartido por los 3 programas** **3**
`struct Puntos` . . . 3
`leerPuntosCSV(ruta)` . . . 3
`escribirResultadoCSV(ruta, puntos, tipo)` . . . 3
`dist2(puntos, i, j)` . . . 3
`struct Parametros` y `parsearArgumentos(argc, argv)` . . . 3

`src/dbscan_serial.cpp` **— versión de referencia** **4**

`src/dbscan_p1.cpp` **— "matriz indivisible"** **4**
Idea central . . . 4
`omp_set_num_threads(par.hilos)` . . . 5
`schedule(dynamic, 64)` — por qué dinámico y por qué 64 . . . 5
Por qué NO hay condiciones de carrera (*race conditions*) . . . 5
La barrera implícita . . . 5

`src/dbscan_p2.cpp` **— "dividiendo la matriz por k"** **5**
Idea central . . . 5
Cómo se calculan los límites de cada bloque . . . 6
El patrón "map + merge" . . . 6
Por qué el merge tiene que terminar ANTES del Paso 2 (la consistencia entre bloques) 6
Cómo se verificó que esto realmente funciona . . . 7

El arnés de experimentos **7**
`src/generar_puntos.py` . . . 7
`experimentos/run_reanudable.sh` . . . 7
`experimentos/analizar.py` . . . 7

Preguntas trampa que podrían hacerles **8**

Propósito de este documento

Este documento existe para que puedan **contestar cualquier pregunta** sobre cualquier parte del código entregado. Recordátorio del enunciado del proyecto: *"Si no se puede explicar a detalle cualquier parte del programa, 80% menos."* No es para leerlo en la presentación —es para tenerlo a la mano y consultarlo si les preguntan algo específico.

El algoritmo, explicado a fondo

Qué es un punto "core" y qué es "ruido"

El algoritmo implementado (el "algoritmo ingenuo" del PDF del proyecto) recibe dos parámetros:

- `epsilon`: un radio de distancia.
- `min_samples`: un número mínimo de vecinos.

Y hace dos pasadas sobre el conjunto de `n` puntos:

Paso 1. Para cada punto `i`, cuenta cuántos puntos `j` (de **todo** el conjunto, `j != i`) cumplen `distancia(i, j) <= epsilon`. Si ese conteo llega a `min_samples`, el punto `i` se marca como **core de primer orden** (`tipo = 1`). Si no, queda temporalmente como "candidato a ruido".

Paso 2. Para cada punto que quedó como candidato a ruido, se revisa si existe **algún** punto core (de los encontrados en el Paso 1) a distancia `<= epsilon`. Si existe, el punto se reetiqueta como **core de segundo orden** (`tipo = 2`). Si no existe ninguno, el punto se queda como **ruido/outlier** (`tipo = 0`).

Noten que este algoritmo **no asigna número de clúster** a cada punto —no dice "este punto pertenece al clúster 3"—, solo clasifica cada punto en una de tres categorías (core 1° orden / core 2° orden / ruido). Eso es exactamente lo que pide el título del proyecto: *"DBSCAN paralelo para la detección de ruido o outliers"*, no clustering completo.

Por qué es O(n²)

El Paso 1 compara cada uno de los `n` puntos contra los otros `n-1` puntos → del orden de `n×n` comparaciones de distancia en el peor caso. El Paso 2 compara cada punto-candidato-a-ruido contra la lista de puntos core (que en el peor caso también puede ser de tamaño `n`). Por eso decimos que el algoritmo es **O(n²)**.

Optimización que SÍ se aplicó (sin cambiar el orden de complejidad): en el Paso 1, en cuanto el conteo de vecinos llega a `min_samples`, se corta el ciclo interno con `break` —ya sabemos que es core, no hace falta seguir contando—. Esto reduce el trabajo real (sobre todo en zonas densas) sin cambiar el peor caso teórico.

Optimización que SÍ se aplicó — distancia al cuadrado: en vez de calcular `sqrt((x2-x1)^2 + (y2-y1)^2) <= epsilon`, se calcula `(x2-x1)^2 + (y2-y1)^2 <= epsilon^2`. Es matemáticamente equivalente (ambos lados son no negativos, así que elevar al cuadrado preserva la desigualdad) y evita calcular una raíz cuadrada por cada comparación —una operación cara, multiplicada por millones de comparaciones—.

`src/dbscan_common.h` — código compartido por los 3 programas

`struct Puntos`

```
struct Puntos {
    std::vector<double> x;
    std::vector<double> y;
    size_t n() const { return x.size(); }
};
```

Guarda las coordenadas de todos los puntos en **dos arreglos paralelos** (`x[i]`, `y[i]` son las coordenadas del punto `i`), en vez de un arreglo de pares `(x,y)`. Esto se llama *structure of arrays* (SoA) en vez de *array of structures* (AoS); es más amigable con la caché del procesador cuando se recorren muchos puntos seguidos, porque las `x` quedan juntas en memoria y las `y` juntas en memoria.

`leerPuntosCSV(ruta)`

Abre el archivo, lee línea por línea. La primera línea se revisa: si no empieza con un dígito, signo o punto, se asume que es la cabecera (`x,y`) y se descarta; si sí parece un número, se asume que no hay cabecera y se procesa como dato. Así el programa acepta CSV con o sin cabecera. Cada línea se separa por coma (`std::getline(ss, a, ',')`) y se convierte a `double` con `std::stod`.

`escribirResultadoCSV(ruta, puntos, tipo)`

Escribe la cabecera `x,y,tipo` y luego una fila por punto, con 10 dígitos de precisión (`out.precision(10)`).

`dist2(puntos, i, j)`

```
inline double dist2(const Puntos& p, size_t i, size_t j) {
    double dx = p.x[i] - p.x[j];
    double dy = p.y[i] - p.y[j];
    return dx * dx + dy * dy;
}
```

Distancia euclidiana **al cuadrado** entre los puntos `i` y `j` (ver la sección de optimizaciones arriba, de por qué no se usa `sqrt`).

`struct Parametros` y `parsearArgumentos(argc, argv)`

Guarda y parsea los argumentos de línea de comandos. Tabla completa:

Flag	Significa	Obligatorio	Usado por
`-i <archivo>`	CSV de entrada (puntos `x,y`)	Sí	los 3 programas
`-o <archivo>`	CSV de salida (`x,y,tipo`)	Sí	los 3 programas
`-e <número>`	`epsilon`, radio de vecindad	No (default 0.03)	los 3 programas

Flag	Significa	Obligatorio	Usado por
`-m <entero>`	`min_samples`, vecinos mínimos	No (default 10)	los 3 programas
`-k <entero>`	número de bloques	No (default 4)	**solo** `dbscan_p2`
`-t <entero>`	número de hilos OpenMP	No (default 1)	`dbscan_p1` y `dbscan_p2` (en `dbscan_serial` se ignora: siempre corre con 1 hilo)
`-q`	no imprimir la línea `TIEMPO ...`	No	los 3 programas

Si falta `-i` o `-o`, el programa lanza un error y termina (`std::runtime_error`).

`src/dbscan_serial.cpp` — versión de referencia

Flujo completo:

1. Parsea argumentos, lee el CSV de entrada → `Puntos p`.
2. `eps2 = epsilon * epsilon` (se calcula una sola vez, fuera de los ciclos).
3. `t0 = omp_get_wtime()` —arranca el cronómetro (sí, se usa `omp_get_wtime()` aunque sea la versión serial, porque es un reloj de alta resolución cómodo, no implica que el código sea paralelo—.
4. **Paso 1**, con un ciclo `for` normal (sin `#pragma omp`) de `i = 0` a `n-1`: cuenta vecinos, corta con `break` al llegar a `min_samples`, marca `tipo[i] = 1` si aplica.
5. Construye el vector `core` con los índices de todos los puntos que quedaron `tipo[i] == 1`.
6. **Paso 2**, otro ciclo `for` normal: para cada punto con `tipo[i] == 0`, recorre `core` y si encuentra uno a distancia `<= eps2`, marca `tipo[i] = 2` y corta con `break`.
7. `t1 = omp_get_wtime()` —para el cronómetro—.
8. Escribe el CSV de salida.
9. Si `medirTiempo` está activo (por default sí, a menos que se pase `-q`), imprime en `stderr`: `TIEMPO serial,<n>,1,1,<segundos>`.

`src/dbscan_p1.cpp` — “matriz indivisible”

Idea central

Se toma exactamente el mismo código del Paso 1 y Paso 2, pero el ciclo externo (el que recorre los `n` puntos) se marca con:

```
#pragma omp parallel for schedule(dynamic, 64)
for (size_t i = 0; i < n; i++) { ... }
```

Esto le dice al compilador y al *runtime* de OpenMP: “reparte las iteraciones de este ciclo entre los hilos disponibles”. **No hay ninguna división manual del arreglo de puntos** —por eso “matriz indivisible”—: el programador solo dice *qué* se puede paralelizar, y OpenMP decide *cómo* repartirlo en tiempo de ejecución.

`omp_set_num_threads(par.hilos)`

Antes de entrar a cualquier región paralela, se fija explícitamente cuántos hilos usar OpenMP (en vez de dejar que use el valor por default del sistema, que normalmente es "todos los núcleos disponibles"). Así el experimento controla exactamente el número de hilos de cada corrida.

`schedule(dynamic, 64)` — por qué dinámico y por qué 64

- `dynamic` (en vez de `static`, que es el default): cada hilo pide un nuevo bloque de iteraciones **cuando termina** las que tenía, en vez de que se le asigne de antemano una porción fija del arreglo. Esto importa porque el costo por punto **no es uniforme**: un punto en zona densa corta rápido con `break` (barato), un punto en zona de ruido recorre todo el arreglo (caro). Con `static`, si por mala suerte todos los puntos caros caen en el mismo hilo, ese hilo se vuelve el cuello de botella mientras los demás ya terminaron. Con `dynamic`, el trabajo se reparte según quién va terminando, balanceando mejor la carga real.
- `64` es el tamaño de "paquete" (*chunk*) que cada hilo pide de un jalón. Pedir de uno en uno tendría demasiado *overhead* de sincronización (cada punto individual implica pedirle trabajo nuevo al *scheduler*); pedir bloques de 64 puntos a la vez reduce ese *overhead* sin perder demasiado balance de carga. Es un valor empírico razonable, no viene de una fórmula exacta.

Por qué NO hay condiciones de carrera (*race conditions*)

Dentro del ciclo paralelo, cada hilo, al procesar el punto `i` que le tocó, **únicamente escribe en** `tipo[i]` —su propia casilla—. Dos hilos nunca procesan el mismo `i` al mismo tiempo (eso lo garantiza OpenMP), así que nunca hay dos escrituras simultáneas a la misma dirección de memoria. Por eso **no hacen falta** secciones críticas (`#pragma omp critical`), candados (*locks*), ni operaciones atómicas (`#pragma omp atomic`).

La única lectura compartida es `p.x`, `p.y` (las coordenadas) y, en el Paso 2, el vector `core`: todos los hilos **leen** esos datos, ninguno los modifica durante la región paralela, así que leer al mismo tiempo es seguro.

La barrera implícita

Al final de un bloque `#pragma omp parallel for`, OpenMP inserta automáticamente una **barrera**: ningún hilo sale de esa región hasta que **todos** los hilos terminaron sus iteraciones. Esto es crítico aquí: el código que construye el vector `core` (recorriendo `tipo[]` buscando `tipo[i]==1`) se ejecuta **después** del `parallel for` del Paso 1, así que, gracias a la barrera, cuando se construye `core` ya están **todos** los puntos core del Paso 1 marcados —ningún hilo se quedó atrás—.

`src/dbscan_p2.cpp` — "dividiendo la matriz por k"

Idea central

A diferencia de P1, aquí el programador divide **a mano** los `n` puntos en `k` bloques contiguos (el parámetro `-k`), **independiente** del número de hilos (`-t`). Puede haber más bloques que hilos (en cuyo caso un hilo procesa varios bloques, uno tras otro) o menos bloques que hilos (en cuyo caso algunos hilos se quedan sin bloque que procesar).

Cómo se calculan los límites de cada bloque

```
std::vector<size_t> inicioBloque(k + 1);
for (int b = 0; b <= k; b++) inicioBloque[b] = (n * (size_t)b) / k;
```

Esto da `k` bloques de tamaño lo más parejo posible (algunos de `n/k` puntos, otros de `n/k + 1`, según si `n` es divisible entre `k`). El bloque `b` va del índice `inicioBloque[b]` al `inicioBloque[b+1]-1`.

El patrón "map + merge"

Paso 1 de P2:

```
std::vector<std::vector<int>> tipoLocalP1(k);
#pragma omp parallel for schedule(dynamic)
for (int b = 0; b < k; b++) {
    // procesa el bloque b COMPLETO en un ciclo normal (secuencial dentro del bloque)
    // escribe en tipoLocalP1[b], un buffer PROPIO de este bloque
}
// MERGE explícito: copia cada buffer local al arreglo global "tipo"
for (int b = 0; b < k; b++) { ... tipo[ini + idx] = tipoLocalP1[b][idx]; ... }
```

Aquí la paralelización es sobre el **índice de bloque** `b` (de 0 a k-1), no sobre el índice de punto `i` como en P1. Cada bloque, mientras se procesa, es dueño exclusivo de su propio `std::vector<int> local` —nadie más lo toca—, así que tampoco hay condiciones de carrera aquí. Pero a diferencia de P1, los resultados **no** quedan directamente en el arreglo global `tipo`: quedan en buffers separados (`tipoLocalP1[0]`, `tipoLocalP1[1]`, ..., `tipoLocalP1[k-1]`), y hace falta un ciclo **después** de la región paralela que copie cada buffer a su lugar correspondiente en `tipo`. Ese ciclo de copiado es el **merge**.

¿Por qué usar buffers separados en vez de escribir directo en `tipo` como en P1? Para este algoritmo en particular, escribir directo también sería seguro (cada bloque toca índices distintos de `tipo`). Se hizo con buffers separados **a propósito pedagógico**: así el patrón queda explícito —"cada bloque calcula su parte de forma aislada, y al final se unen los resultados"—, que es el mismo patrón que se tendría que usar si los bloques vivieran en **procesos distintos o máquinas distintas** (como pasaría con MPI, el siguiente tema del curso), donde sí sería obligatorio tener resultados locales y después mandarlos/juntarlos.

Por qué el merge tiene que terminar ANTES del Paso 2 (la consistencia entre bloques)

Esta es la pregunta explícita que hace el PDF del proyecto ("¿Cómo unir los resultados para mantener consistencia?"). La respuesta:

- El Paso 1 de cada bloque **ya es correcto de forma aislada**: cada punto del bloque se compara contra **todo** el arreglo global de puntos (no solo contra su propio bloque), así que no importa en qué bloque haya quedado un punto, su clasificación de core-de-primer-orden es la misma que si se hubiera calculado sin dividir en bloques.
- El problema aparece en el Paso 2: un punto del **bloque A** puede convertirse en core-de-segundo-orden gracias a un punto core que encontró el **bloque B** en SU Paso 1. Si el bloque A empezara su Paso 2 antes de que el bloque B terminara su Paso 1 (y se mergeara), el bloque A no sabría que ese punto core de B existe, y clasificaría mal ese punto como ruido.
- Por eso, entre el Paso 1 y el Paso 2 hay **dos cosas obligatorias, en este orden**: (1) la barrera implícita del `#pragma omp parallel for` del Paso 1 (espera a que todos los bloques terminen su Paso 1), y (2) el ciclo de merge que copia **todos** los buffers locales al arreglo

global `tipo` **antes** de construir la lista `core` que usa el Paso 2. Solo entonces arranca el Paso 2 (que, de nuevo, se procesa por bloques con su propio merge final).

Cómo se verificó que esto realmente funciona

Se corrieron las tres versiones (serial, P1, P2) sobre el mismo conjunto de 2,000 puntos de prueba, y se comparó el conteo de cada categoría (`0`=ruido, `1`=core 1er orden, `2`=core 2do orden) en el CSV de salida. Los tres dieron **exactamente los mismos conteos** (264 ruido, 1,691 core 1er orden, 45 core 2do orden), lo que confirma que el particionamiento en bloques y el merge no introducen errores de clasificación.

El arnés de experimentos

`src/generar_puntos.py`

Genera puntos aleatorios en 2D: 85% en var ios cúmulos gaussianos (simulando zonas densas) y 15% de ruido uniforme en todo el cuadrado de `100×100`. Se guarda con semilla fija (`--semilla 42` en los experimentos) para que el mismo archivo de entrada se use en todas las corridas de un mismo `n` (comparación justa entre versiones).

Actualización (17 de octubre de 2026): se localizó y verificó la libreta oficial `DBSCAN_noise.ipynb` en el repositorio del curso (github.com/octavio-gutierrez/computoparalelo2026b). El script se reescribió para usar **exactamente** el mismo método: `sklearn.datasets.make_blobs(n_samples=n, centers=4, cluster_std=0.06, random_state=11, center_box=(0,1.0))`, seguido de `np.round(np.abs(points[:, ::-1]), 3)` y `np.savetxt(..., delimiter=",", fmt="%.3f")` sin encabezado. A diferencia de la versión anterior, ya **no se agrega ruido uniforme a mano**: el ruido que detecta DBSCAN surge naturalmente de la cola dispersa de cada cúmulo gaussiano, igual que en la libreta oficial. Por el cambio de escala (puntos en el rango ~[0, 1.3] en vez de [0, 100]), se ajustaron también `epsilon=0.03` y `min_samples=10` (mismos valores que usa la libreta), y se volvió a correr el experimento completo con estos parámetros.

`experimentos/run_reanudable.sh`

Corre la rejilla completa de experimentos: para cada combinación de `(versión, n, k, hilos)`, revisa cuántas de las 10 iteraciones ya están en `datos/resultados.csv` y solo corre las que faltan. Esto lo hace **reanudable**: si se interrumpe a la mitad, la siguiente vez que se corre retoma donde se quedó en vez de repetir todo desde cero. Se construyó así porque, al correr 2,320 experimentos, el proceso tarda varios minutos y conviene poder pausarlo/reanudarlo sin perder avance.

`experimentos/analizar.py`

Lee `datos/resultados.csv`, calcula el promedio y la desviación estándar de las 10 iteraciones de cada configuración (`datos/resultados_promedio.csv`), calcula el *speedup* de cada configuración paralela contra el tiempo serial del mismo `n` (`datos/speedups.csv`: `speedup = tiempo_serial / tiempo_version`), y genera las 5 gráficas en `graficas/`.

Preguntas trampa que podrían hacerles

- **“¿Qué pasa si dos hilos intentan leer `p.x[j]` al mismo tiempo?”** — Es seguro: leer la misma posición de memoria desde varios hilos simultáneamente **no** es una condición de carrera (solo lo es cuando al menos una de las operaciones es una escritura). Nadie modifica `p.x`/`p.y` durante la región paralela.
- **“¿Por qué no usaron `#pragma omp critical` en ningún lado?”** — Porque no hace falta: cada hilo (P1) o cada bloque (P2) escribe únicamente en su propia posición/buffer, así que no hay recurso compartido que proteger.
- **“¿El resultado depende del número de hilos o del valor de k?”** — No debería, y se verificó que no: la clasificación final de cada punto es idéntica sin importar cuántos hilos o bloques se usen —eso es justo lo que hace “correcta” a la paralelización—.
- **“¿Qué pasa si `k=1` en P2?”** — Se procesa como un solo bloque con todos los puntos; en teoría debería comportarse muy parecido a la versión serial pero usando solo 1 de los hilos disponibles (el resto no tiene bloque que procesar).
- **“¿Qué significa que el speedup sea menor a 1.0x con 1 hilo?”** — Que la versión “paralela” con un solo hilo es más lenta que la serial pura; se debe al *overhead* de inicializar la región paralela de OpenMP (y, en P2, de crear los buffers locales) sin obtener ningún beneficio real de paralelismo.
- **“¿Cómo midieron el tiempo exactamente?”** — Con `omp_get_wtime()` (un reloj de pared de alta resolución que da OpenMP), puesto **antes** del Paso 1 y **después** del Paso 2, sin incluir la lectura ni la escritura de archivos CSV —solo el cómputo—.